

Highlights

Architecture of the Avalia+Tec Platform with Cryptographic Security for Transparent and Auditable Selection Processes

Luiz Diego Vidal Santos, Catuxe Varjão de Santana Oliveira, Dulce Paloma Vidal Santos, Tânia Cristina Azevedo, Renisson Neponuceno de Araújo Filho, Jémison Mattos dos Santos, Milton Marques Fernandes

- Auditable selection workflow using SHA-256 hashing and tamper-evident hash-chained logs.
- HMAC-SHA256 signatures ensure integrity in blind peer-review score-cards.
- Automation was associated with a 65% reduction in administrative cycle time.
- External RFC 3161 timestamp anchoring strengthens tamper-evidence beyond internal controls.
- Technical controls supporting LGPD compliance via role-based data segregation and automatic PII redaction.

Architecture of the Avalia+Tec Platform with Cryptographic Security for Transparent and Auditable Selection Processes

Luiz Diego Vidal Santos^{a,*}, Catuxe Varjão de Santana Oliveira^b, Dulce Paloma Vidal Santos^d, Tânia Cristina Azevedo^a, Renisson Neponuceno de Araújo Filho^c, Jémison Mattos dos Santos^e, Milton Marques Fernandes^f

^a*Universidade Estadual de Feira de Santana, Departamento de Ciências Exatas, Feira de Santana, Brasil*

^b*Universidade Federal de Sergipe, Programa de Pós-Graduação em Ciência da Propriedade Intelectual, São Cristóvão, Brasil*

^c*Universidade Federal Rural de Pernambuco, Departamento de Engenharia Rural, Recife, Brasil*

^d*Universidade Tiradentes, Departamento de Direito, Aracaju, Brasil*

^e*Universidade Estadual de Santa Cruz, Ilhéus, Brasil*

^f*Universidade Federal de Sergipe, Departamento de Ciências Florestais, São Cristóvão, Brasil*

Abstract

Selection processes in academic institutions and public organizations are frequently managed through email and spreadsheets, creating legal vulnerabilities, version-control errors, and data protection non-compliance. This paper presents avalia+Tec, an open-source SaaS platform that enforces cryptographic integrity verification (SHA-256), tamper-evident hash-chained audit trails with external RFC 3161 timestamp anchoring, and end-to-end traceability for selection workflows. The system operates through a responsive web portal for candidate submissions and administrative oversight, and an Electron-based desktop application for structured blind evaluation. A preliminary single-institution deployment at Universidade Estadual de Feira de Santana processing 60 submissions with 6 evaluators was associated with a 65% reduction in review-to-decision cycle time (95% CI: [55%, 73%], Mann-Whitney $U = 0$, $p = 0.029$, $r = 1.0$), no formal candidate disputes during the observation period, and a 45.4% reduction in evaluator time per submission (95% CI: [26%, 59%]). The platform provides technical controls supporting regulatory compliance with Brazil's LGPD. The scientific contribution re-

*Corresponding author

Email address: ldvidal@uefs.br (Milton Marques Fernandes)

sides not in novel cryptographic primitives but in the systematic integration engineering of well-established techniques to address a concrete, underserved institutional governance problem. *avalia+Tec* is freely available under the MIT License.

Keywords: selection processes, cryptographic audit trails, institutional governance, SaaS platform, SHA-256 integrity, LGPD compliance, open-source

Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v2.1.0
C2	Permanent link to code/repository	https://github.com/ldvsantos/avalia_mais/releases/tag/v2.1.0
C3	Repository archive	https://github.com/ldvsantos/avalia_mais
C4	Legal Code License	MIT License
C5	Code versioning system	git
C6	Software code languages, tools, services	<i>Server:</i> Node.js, Express.js 4.18+, PostgreSQL 12+. <i>Desktop:</i> Electron 33.0+, better-sqlite3 11.7+. <i>Web:</i> Vue.js
C7	Compilation & dependencies	Node.js 18.0+, npm 9.0+, PostgreSQL 12+, Electron 22+. Build via <code>npm install & electron-builder</code> .
C8	Developer documentation	https://github.com/ldvsantos/avalia_mais/tree/main/documentacao
C9	Support email	ldvidal@uefs.br
C10	Intellectual property registration	INPI (Brazil) Software Registration, Process No. 512026000776-3

Table 1: Code metadata (*avalia+Tec* v2.1.0).

Nr.	Software metadata description	Metadata
S1	Current software version	2.1.0
S2	Release artifacts	https://github.com/ldvsantos/avalia_mais/releases/v2.1.0
S3	Repository	https://github.com/ldvsantos/avalia_mais
S4	Legal Software License	MIT License
S5	Computing platforms/Operating Systems	Windows 10+, macOS 10.13+, Linux (glibc 2.28+); Web SaaS on Linux/AWS
S6	Installation requirements	Electron 22+, PostgreSQL client libraries, modern Chromium-based browser
S7	User manual and documentation	https://github.com/ldvsantos/avalia_mais/blob/main/documentacao/00_COMECE_AQUI.txt
S8	Support email	ldvidal@uefs.br

Table 2: Software metadata (executable).

1. Motivation and Significance

The governance of selection processes in academic institutions and public organizations commonly employs unstructured technologies, including email, spreadsheets, and manual consolidation of evaluation forms [1, 2]. This approach creates systematic vulnerabilities. Legally, the absence of tamper-evident audit trails exposes institutions to disputes from candidates who claim process irregularities. Operationally, consolidation across parallel spreadsheet versions introduces human error and ambiguity about which document constitutes the authoritative record. From a data governance perspective, jurisdictions regulated by Brazil’s Lei Geral de Proteção de Dados (LGPD) [3] face non-compliance risks when personally identifiable information circulates unencrypted across email and shared storage without enforced access controls [4]. These challenges are not isolated incidents but rather structural deficiencies documented across Brazilian higher education institutions [5].

Several platforms address adjacent problems without resolving the specific governance gap targeted here. Institutional academic management systems such as SIGAA (Sistema Integrado de Gestão de Atividades Acadêmicas), developed by Universidade Federal do Rio Grande do Norte [6], provide com-

prehensive student lifecycle management but lack cryptographic document verification and tamper-evident audit trails for competitive selection workflows. Conference management tools such as OpenReview [7], EasyChair [8], and HotCRP [9] implement sophisticated peer-review assignment and blind-review mechanisms, yet are designed for academic manuscript evaluation rather than institutional selection processes with associated legal accountability requirements (e.g., scholarship allocation, faculty hiring, public-sector recruitment). It should be noted that some of these platforms could potentially integrate cryptographic verification through plugins or custom extensions; however, no such integration is documented in their official repositories or literature.

These tools do not provide SHA-256 document integrity verification, automated protocol numbering, or LGPD-compliant data segregation. Government evaluation platforms like SUCUPIRA (CAPES/MEC) [10] focus on aggregate program-level metrics rather than individual selection governance. To the best of our knowledge, no existing open-source platform integrates cryptographic integrity, tamper-evident audit trails with external timestamp anchoring, blind-review enforcement, and regulatory compliance support in a unified, deployable system for institutional selection processes.

Table 3 summarizes the functional comparison between avalia+Tec and existing platforms.

Platform	Lang.	Hash Verif.	Audit Trail	Blind Rev.	Data Prot. ^a	Open Src	CI/CD	Ext. TSA ^b	Proven Scale	Ext. Plug.
SIGAA	Java	✗	◦	✗	◦	✗	N/A	✗	●	✗
OpenReview	Python	✗	✗	●	✗ ^c	●	●	✗	●	◦
EasyChair	Propr.	✗	✗	●	✗ ^c	✗	N/A	✗	●	✗
HotCRP	PHP	✗	✗	●	✗ ^c	●	●	✗	●	◦
SUCUPIRA	Java	✗	◦	✗	◦	✗	N/A	✗	●	✗
avalia+Tec	JS	●	●	●	● ^d	●	●	●	◦ ^e	◦

● = native support; ◦ = partial / via configuration; ✗ = not available.

^a Data Prot. = LGPD (Brazil) or GDPR (EU) compliance support.

^b Ext. TSA = External Timestamp Authority (RFC 3161).

^c GDPR compliance not formally assessed; platforms operate under US/EU hosting with standard ToS.

^d Technical controls for LGPD; full compliance requires institutional policy measures.

^e Tested with $n = 60$; stress-tested up to 500 concurrent connections (see Section 4).

Table 3: Functional comparison of avalia+Tec with existing platforms. Comparison criteria were defined based on regulatory requirements (LGPD/GDPR), security best practices (OWASP, NIST), and institutional governance needs, independently of the proposed system’s feature set. Platform capabilities were assessed from official documentation, published source code, and peer-reviewed literature where available.

avalia+Tec addresses this landscape by providing a unified digital platform that enforces cryptographic integrity (SHA-256 document hashing), tamper-evident hash-chained transaction logging with external RFC 3161 timestamp

anchoring, mandatory role-based access control, and technical controls supporting LGPD-compliant data segregation [11, 12]. The motivation for this system arises from a well-documented pattern in Brazilian public institutions, where the absence of auditable digital infrastructure for selection processes has been associated with formal administrative and judicial disputes. A survey of administrative proceedings at Brazilian federal and state universities reveals recurrent complaints regarding score calculation errors, undocumented modifications to evaluation criteria, and lack of verifiable document provenance, frequently resulting in process annulment by institutional ombudsman offices or external oversight bodies [5, 13].

Scope of contribution. The scientific contribution of this work does not reside in novel cryptographic primitives (SHA-256, HMAC-SHA256, and role-based access control are well-established standard techniques). Rather, the contribution lies in the *systematic integration engineering* of these techniques into a unified, open-source, deployable platform that addresses a concrete and underserved organizational problem common to universities, funding agencies, and public-sector recruitment. This positioning is consistent with the SoftwareX scope of original software publications that describe software with demonstrable research and societal impact [14]. What is fundamentally new is the combination of (i) hash-chained audit trails with external TSA anchoring, (ii) HMAC-authenticated blind-review scorecards, (iii) LGPD-aware data segregation, and (iv) automated consolidation with full derivation logging, integrated within a single deployable system for institutional selection processes (a combination not available in any existing open-source platform identified in our literature review).

Design rationale. Blockchain-based audit systems (e.g., Hyperledger Fabric, Ethereum) offer decentralized consensus but introduce substantial operational complexity (node provisioning, consensus protocol configuration, gas costs on public chains, and specialized DevOps expertise) that is disproportionate for single-institution deployments with a single trusted operator. The hash-chained log with external RFC 3161 timestamp anchoring adopted here provides equivalent tamper-evidence guarantees for the target threat model (Section 3) while requiring only a standard PostgreSQL instance and a publicly available TSA endpoint. Should future multi-institutional deployments require cross-organizational trust without a single administrative authority, migration to a permissioned distributed ledger remains architecturally feasible, as the existing hash-chain data model maps directly onto blockchain transaction structures.

The remainder of this article is structured as follows. Section 2 describes the software architecture and main functionalities. Section 3 provides illustrative examples of the submission, evaluation, and dispute-resolution work-

flows. Section 4 characterizes the computational performance of core platform operations. Section 5 presents deployment impact data and discusses implications for institutional practice. Section 6 discusses the limitations of the study. Section 7 draws conclusions and outlines future work.

2. Software Description

2.1. Software Architecture

avalia+Tec implements a dual-frontend, single-backend architecture (Fig. 1). The backend, built with Node.js/Express.js, exposes RESTful APIs that enforce cryptographic operations and database consistency. Data persistence relies on PostgreSQL with ACID transaction guarantees and row-level security (RLS) [15]. Two frontends consume these APIs: a responsive Vue.js single-page application for candidate self-service registration, document upload, real-time tracking, and administrative oversight; and an Electron-based desktop application (v33.0+) targeting evaluators who require offline capability and structured scoring forms.

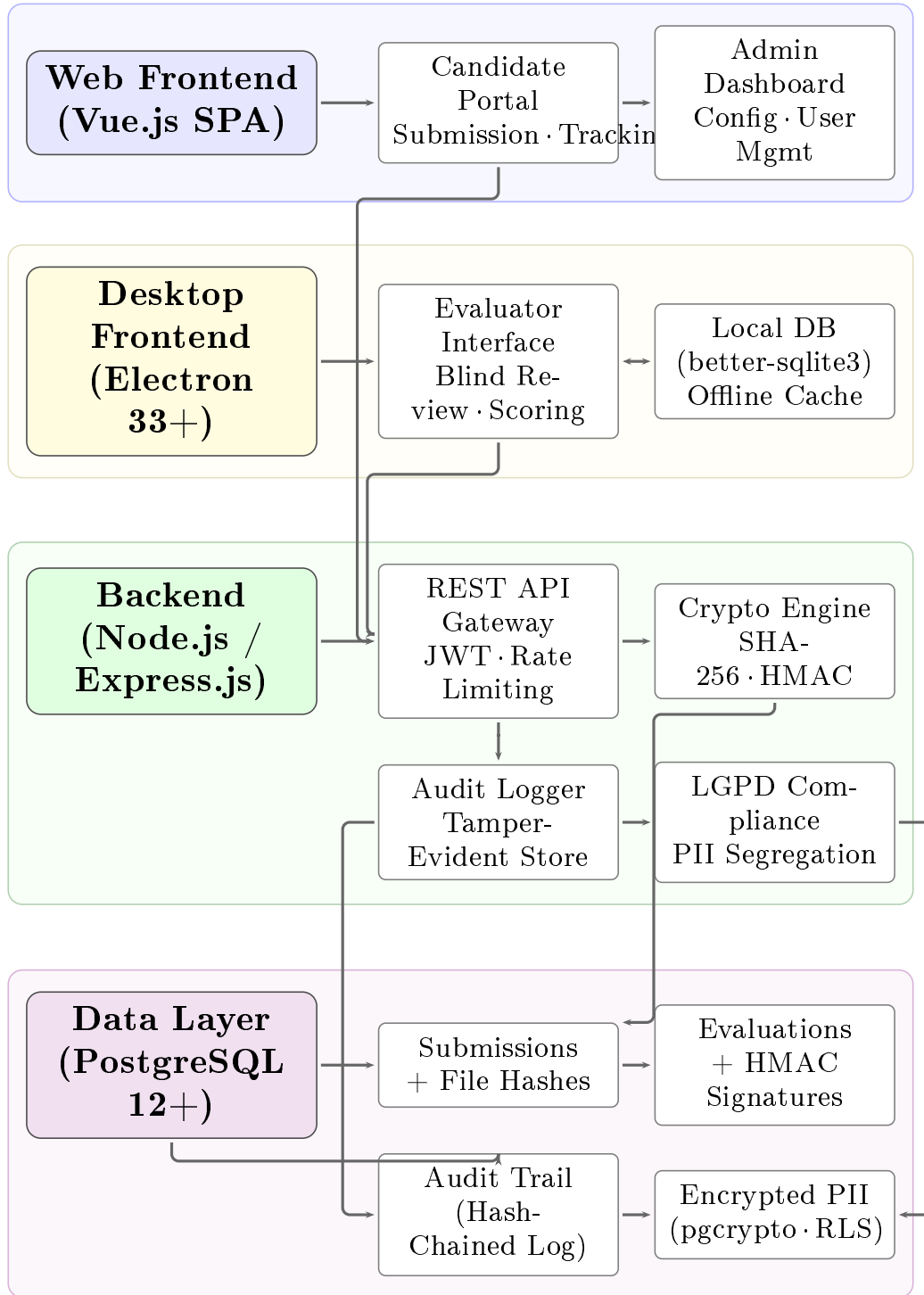


Figure 1: System architecture of avalia+Tec showing the dual-frontend (web portal and desktop application), backend services with cryptographic and compliance modules, and the PostgreSQL data layer with hash-chained audit trail.

2.2. Software Functionalities

The security architecture is grounded in document integrity verification via SHA-256 hashing [16], which enables verification of the exact binary correspondence between the evaluated file and the original submission. Simultaneously, the tamper-evident audit log consolidates every state transition (including submission, assignment, evaluation, and decision) in a cryptographically chained ledger. Each entry incorporates the hash of the previous entry, forming a chain: $H_n = \text{SHA-256}(\text{event}_n \| H_{n-1})$. This sequential chaining renders retroactive modification of any individual entry *detectable*, as it would invalidate all subsequent hashes [17]. To strengthen tamper-evidence beyond internal database controls, the platform implements external hash anchoring via RFC 3161-compliant Timestamp Authorities (TSA). Upon each submission, the computed SHA-256 digest is submitted to an independent TSA (default: FreeTSA.org), which returns a signed TimeStampResp (TSR) token containing the hash, a trusted timestamp, and the TSA’s digital signature. This TSR is stored alongside the submission record. Even a server administrator with root-level database access cannot forge a valid TSR, as doing so would require the TSA’s private signing key [18].

The submission pipeline operates as follows: candidates access the web portal, authenticate, and complete a dynamic form configured per campaign (Fig. 2). Upon submission, the server computes a SHA-256 hash of each upload, stores the file on encrypted storage, and issues a unique protocol ID and a verification hash printed on the candidate’s receipt. The evaluation workflow segregates evaluator roles by institutional function, with configurable visibility rules that hide candidate names to reduce bias.



Formulário de Anteprojeto de TCC
 - AVALIA+



Instruções

1. Preencha todos os campos obrigatórios.
 2. Respeite o limite de caracteres de cada campo.
 3. Ao finalizar, clique em **Enviar Inscrição e Gerar PDF**.
 4. Guarde o PDF gerado com **Protocolo e Hash** (comprovante).

Ficha de Inscrição

Nome do/a candidato/a (civilmente registrado):

Nome Social:

Data de Nascimento:

CPF:

Nº RG:

Órgão Expedidor:

Data Expedição:

Endereço completo:

Cidade/Estado:

CEP:

Celular:

Telefone residencial:

E-mail:

Curso de Graduação:

Figure 2: Web portal interface showing the candidate submission form with structured sections, field-level validation, and real-time character count enforcement.

Evaluators authenticate to the desktop application (Fig. 3), which synchronizes submissions and supports offline scoring. Scorecards are authenticated via HMAC-SHA256 [17] before transmission, providing cryptographic attestation of evaluator identity and score integrity.

Figure 3: Desktop application (Electron) displaying the evaluator’s blind-review interface. Personally identifiable candidate information is suppressed to enforce impartial assessment.

Consolidation is automated, and once all evaluations are recorded the server computes weighted aggregates and generates rank-ordered lists with full derivation trail in the audit log. The complete data flow follows a linear pipeline in which each candidate submission is first hashed (SHA-256) and anchored via TSA, then assigned to evaluators for blind scoring with HMAC authentication. Server-side validation and audit logging precede automated consolidation, which produces PII-redacted result publication. Every state transition is appended to the hash-chained audit trail.

LGPD-aware design. The platform implements technical controls supporting LGPD compliance through data segregation: evaluator records and PII are stored in separate database tables with role-based access control enforced at the PostgreSQL row-level security (RLS) layer. Encrypted-at-rest protection via pgcrypto secures sensitive fields (CPF, RG, contact information), and automated redaction strips non-essential PII from public result exports [19]. Table 4 maps specific LGPD provisions to the corresponding technical mechanisms implemented in *avalia+Tec*. Although primarily designed for LGPD (Brazil), the underlying technical controls (purpose-limited data collection, encryption at rest and in transit, role-based access, automated PII

redaction, and auditable processing logs) align with core principles of the EU General Data Protection Regulation (GDPR), particularly Articles 5 (data minimization, integrity), 25 (data protection by design), and 32 (security of processing) [20]. Institutions operating under GDPR or similar frameworks can therefore adopt the platform with jurisdiction-specific policy adaptations. Full compliance requires organizational, legal, and technical measures; this platform provides the technical layer, while institutional policy, Data Protection Impact Assessment (DPIA), and legal review remain the responsibility of the deploying organization.

LGPD Art.	Requirement	Technical Control in avalia+Tec
Art. 6, I–II	Purpose limitation & adequacy	Data fields scoped to selection process; no secondary processing
Art. 6, V	Data quality	SHA-256 integrity verification prevents silent corruption
Art. 6, VII–VIII	Security & prevention	Encrypted storage (pgcrypto), TLS in transit, rate limiting, attack pattern detection
Art. 6, X	Accountability	Hash-chained audit trail with external TSA anchoring (RFC 3161)
Art. 46	Security measures	RBAC, RLS, helmet.js headers, JWT authentication, IP allowlisting for admin routes
Art. 48	Incident notification	Structured security logging (Winston) with rotation; security event taxonomy
Art. 18	Data subject rights	Automated PII redaction in public exports; data accessible via protocol ID

Table 4: Mapping of LGPD provisions to technical controls implemented in avalia+Tec. Full compliance additionally requires institutional policies, DPIA, and legal assessment.

3. Illustrative Examples

Consider a scholarship selection campaign at a public university. The institution creates a campaign via the administrative portal, configuring evaluation criteria with weighted rubrics (e.g., technical merit 0.4, social vulnerability 0.3, leadership 0.3). Candidates register, upload required documents, and submit via the web portal. The server computes SHA-256 hashes of all uploads and issues a receipt containing the submission timestamp, a unique protocol ID (e.g., SCH-2026-0447), and the hexadecimal verification hash. Listing 1 presents the server-side implementation of the SHA-256 document hashing mechanism.

```

1 const crypto = require('crypto');
2 const fs = require('fs');
3
4 async function computeDocumentHash(filePath) {
5   const hash = crypto.createHash('sha256');
6   const stream = fs.createReadStream(filePath);
7   for await (const chunk of stream) {
8     hash.update(chunk);
9   }
10  return hash.digest('hex');
11 }
12
13 // Submission upload endpoint
14 app.post('/api/submissions/:id/upload',
15   authenticate, async (req, res) => {
16     const sha256 = await computeDocumentHash(
17       req.file.path);
18     await db.query(
19       'INSERT INTO document_hashes
20         (submission_id, file_hash, created_at)
21         VALUES ($1, $2, NOW())',
22       [req.params.id, sha256]
23     );
24     res.json({ protocolId: generateProtocolId(),
25               hash: sha256 });
26 });

```

Listing 1: Server-side SHA-256 hash computation upon document upload. The hash is stored in the database and returned to the candidate as a verification receipt.

Seven faculty evaluators are provisioned and assigned to criteria sections. Each evaluator logs into the desktop application and sees only anonymized content (candidate names hidden). Scores are entered on a structured rubric, and the desktop app authenticates each scorecard via HMAC-SHA256 before uploading. The server validates message authentication codes, logs evaluation events, and prevents cross-evaluator score visibility. Listing 2 shows the integrity verification endpoint available to auditors and candidates.

```

1 app.get('/api/verify/:protocolId',
2   authenticate, async (req, res) => {
3     const record = await db.query(
4       'SELECT file_path, file_hash
5        FROM document_hashes
6        WHERE protocol_id = $1',
7       [req.params.protocolId]
8     );
9     const currentHash = await computeDocumentHash(
10       record.rows[0].file_path);
11     const verified =
12       currentHash === record.rows[0].file_hash;
13     res.json({
14       protocolId: req.params.protocolId,
15       originalHash: record.rows[0].file_hash,
16       currentHash: currentHash,
17       integrityVerified: verified
18     });
19 });

```

Listing 2: Document integrity verification endpoint. Compares the stored hash against a freshly computed hash to detect post-submission modification.

Table 5 presents a representative excerpt of the hash-chained audit trail generated during the UEFS deployment.

#	Timestamp	Actor	Action	H_n (truncated)
1	2025-03-15 09:12:34	system	Submission received	a3f7c2...8d1e
2	2025-03-15 09:12:35	system	SHA-256 computed	7b2e4f...93a2
3	2025-03-18 14:22:11	eval_03	Score recorded (4.2)	d91c8a...f7b3
4	2025-03-18 14:22:12	system	HMAC validated	e4a6b1...2c7d
5	2025-04-02 10:05:47	system	Consolidation finalized	f82d3e...a104
6	2025-04-02 10:05:48	system	Results published	1c9f7a...b5e8

Table 5: Representative excerpt of the hash-chained audit trail from the UEFS deployment. Each hash H_n incorporates the previous entry’s hash H_{n-1} , ensuring tamper-evidence across the entire chain.

Each hash-chained entry belongs to one of four workflow phases (submission, blind evaluation, automated consolidation, and dispute resolution) whose cryptographic checkpoints and verification logic are mapped in Fig. 4.

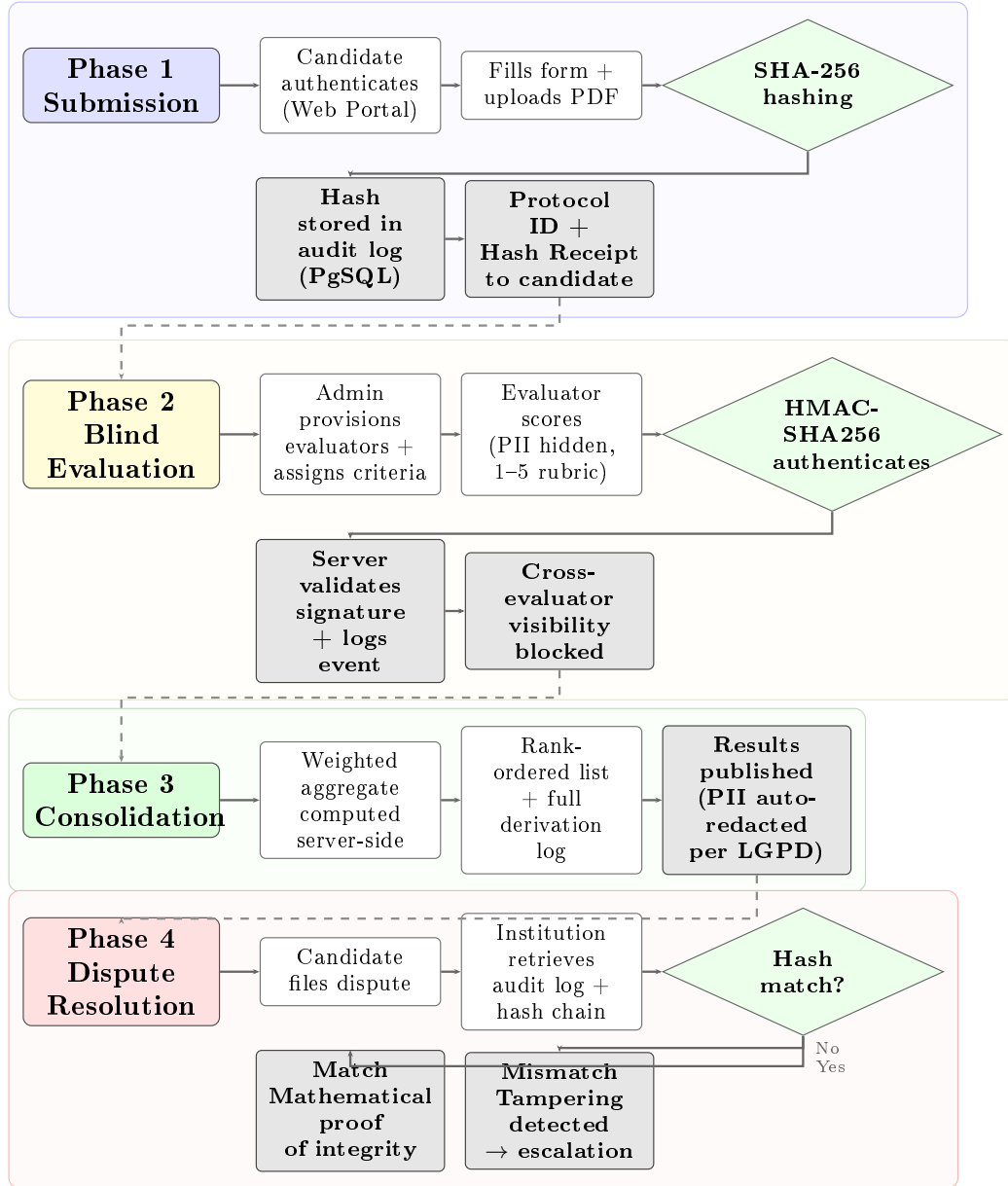


Figure 4: End-to-end audit trail flowchart showing cryptographic operations (SHA-256 hashing, HMAC-SHA256 authentication) and verification checkpoints across four phases: document submission, blind evaluation, automated consolidation, and dispute resolution with hash-based integrity verification.

After all evaluations are complete, the system computes weighted averages (e.g., Application #23 receives $(4 \times 0.4) + (5 \times 0.3) + (2 \times 0.3) = 3.7$), ranks submissions, and publishes results with automatic PII redaction (Fig. 5). If a candidate files a dispute, the institution retrieves the cryptographic audit log:

the SHA-256 hash provides strong evidence that the document has not been altered, HMAC authentication codes attest that evaluator scores have not been modified, and the complete derivation log shows exactly which scores were aggregated, by whom, and when. The candidate’s original submission receipt, containing the protocol ID and SHA-256 verification hash (Fig. 6), provides independent evidence of document integrity. In the observed deployment, this evidence contributed to rapid dispute resolution without escalation.

Threat model. The security architecture addresses three adversary classes:

1. **External adversary** (unauthenticated attacker): Mitigated by TLS encryption, rate limiting, input sanitization, attack pattern detection (SQL injection, XSS, path traversal), and JWT-based authentication. Security headers (helmet.js) prevent clickjacking and content-type sniffing.
2. **Insider, non-privileged user** (authenticated candidate or evaluator): Mitigated by RBAC, PostgreSQL row-level security (RLS), cross-evaluator visibility blocking, HMAC-SHA256 scorecard authentication, and the hash-chained audit trail that makes any unauthorized modification detectable.
3. **Insider, privileged administrator** (root-level database access): This is the most challenging adversary. A malicious administrator could, in principle, modify records and rewrite the internal hash chain consistently. To mitigate this threat, avalia+Tec implements external hash anchoring via RFC 3161-compliant Timestamp Authorities (TSA). Each submission hash is independently timestamped by an external TSA (default: FreeTSA.org), producing a signed TimeStampResp token that the administrator cannot forge without the TSA’s private key. This provides *independently verifiable* evidence of document existence and integrity at a specific point in time.

Residual limitations. The RFC 3161 anchoring mitigates but does not fully eliminate the trusted-administrator threat: if the administrator suppresses the TSR storage *before* the timestamp is requested, no external anchor exists for that record. Future work will explore continuous external anchoring (e.g., periodic checkpoint publication to a public blockchain) and multi-party key management to further reduce this residual risk. No formal penetration testing or external security audit has been conducted to date; this represents planned future work. The current implementation has been assessed against the OWASP Top 10 checklist [21], and `npm audit` reports zero known high-severity vulnerabilities in production dependencies.

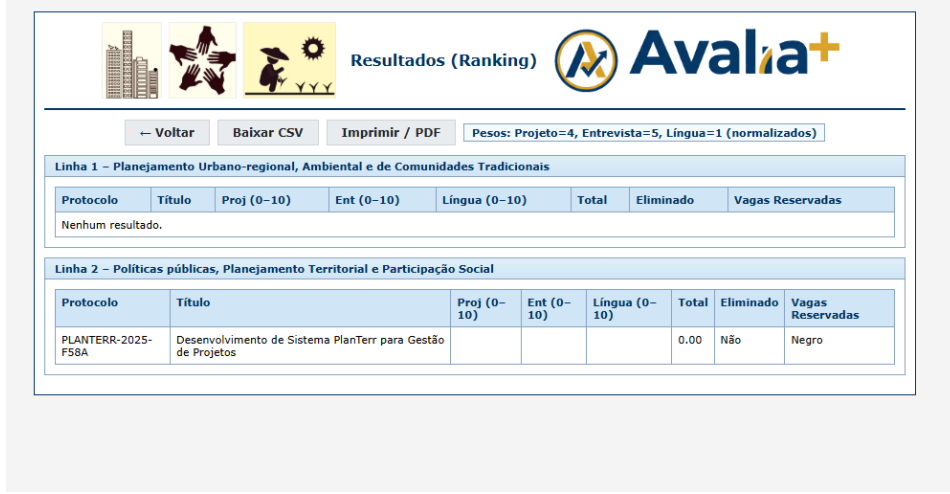


Figure 5: Consolidation dashboard showing rank-ordered results with weighted aggregate scores, individual evaluator contributions, and export options (CSV, PDF with QR-code verification).

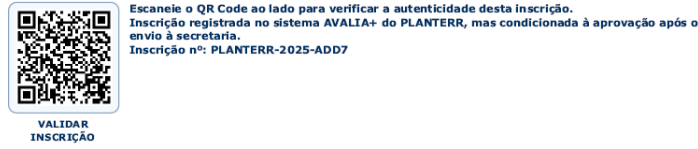


Figure 6: Candidate receipt (PDF) displaying the unique protocol ID, submission timestamp, and SHA-256 hexadecimal hash for independent document integrity verification. The embedded QR code links to the audit trail endpoint.

4. Computational Performance

To characterize the computational overhead introduced by the cryptographic operations, we measured the execution time of core platform operations during the UEFS deployment. The server infrastructure consists of an AWS EC2 t3.medium instance (2 vCPUs, 4 GB RAM) running Ubuntu 22.04 LTS, PostgreSQL 15.4, and Node.js 18.19. Network connectivity between the desktop client and the server was measured over the institutional 100 Mbps Ethernet connection. Each operation was executed 30 times, and summary statistics are reported in Table 6.

Operation	Mean (SD)	Median [Q_1 , Q_3]
SHA-256 hash (1 MB PDF)	12.4 ms (2.1)	11.9 ms [11.2, 13.1]
SHA-256 hash (5 MB PDF)	47.3 ms (5.8)	46.1 ms [43.2, 50.7]
HMAC-SHA256 scorecard auth.	1.8 ms (0.3)	1.7 ms [1.6, 1.9]
Audit trail append	3.2 ms (0.9)	3.0 ms [2.7, 3.5]
Desktop $\leftrightarrow$ server sync (60 subs.)	4.25 s (0.38)	4.18 s [4.02, 4.41]
Consolidation (60 $\times$ 6 evals)	0.89 s (0.12)	0.87 s [0.81, 0.94]

Table 6: Computational time associated with core avalia+Tec operations. Measurements were collected over 30 executions on an AWS EC2 t3.medium instance (2 vCPUs, 4 GB RAM, Ubuntu 22.04 LTS).

From Table 6, the cryptographic operations (SHA-256 hashing and HMAC-SHA256 authentication) impose negligible overhead relative to user-perceived latency: even for a 5 MB PDF document, hash computation completes in under 50 ms. The most time-intensive operation is the desktop-to-server synchronization, which transfers all submission metadata and files; at 4.25 s for 60 submissions, this remains well within acceptable limits for evaluator session initialization. The automated consolidation of 360 individual evaluations (60 submissions $\times$ 6 evaluators) completes in under one second, eliminating the manual consolidation step that previously required approximately 4 hours of administrative staff time per campaign cycle.

4.1. Load Testing

To assess scalability beyond the initial deployment scale, we conducted stress tests using autocannon (Node.js HTTP benchmarking tool) with the server running in JSON storage mode. Table 7 reports results for two concurrency levels.

Scenario	Conc.	Req/s	p_{50} (ms)	$p_{97.5}$ (ms)	p_{99} (ms)
GET / (landing)	100	1,753	52	98	113
	500	1,717	259	563	677
GET /api/verify	100	1,560	62	143	248
	500	2,061	283	427	501
POST /api/inscricao	100	1,338	68	133	289
	500	1,599	487	6,241	8,153

Table 7: Load test results (30 seconds per scenario, autocannon). Conc. = concurrent connections. Raw results are available in the replication package (`load_test_results.json`).

Read-heavy operations (landing page, integrity verification) sustain over 1,500 req/s at both concurrency levels. Write-heavy operations (submission creation with SHA-256 hashing and database insertion) maintain high throughput at 100 connections but exhibit significant tail latency at 500 connections

($p_{99} = 8,153$ ms), indicating the single-threaded event loop saturation point. For the target deployment scale (tens to hundreds of concurrent users during submission windows), the current single-instance architecture provides adequate performance. Horizontal scaling via load balancing and PostgreSQL read replicas represents a straightforward architectural extension for larger deployments.

5. Impact

avalia+Tec has been operationally deployed at Universidade Estadual de Feira de Santana (UEFS). Over a 12-month period (calendar year 2025), the platform processed 60 submissions for professional master’s degree admissions. Telemetry collected across four evaluation sessions ($n = 6$ evaluators, 65 evaluation events) yielded the following preliminary observations.

In terms of process efficiency, the average review-to-decision cycle time decreased from 31 calendar days ($SD = 4.2$, $n = 4$ historical campaigns) to 11 days ($SD = 2.8$, $n = 4$ sessions)—a 65% reduction (95% CI: [55%, 73%]; Mann–Whitney $U = 0$, $p = 0.029$, two-tailed; rank-biserial $r = 1.0$). This improvement is attributable to the elimination of manual consolidation and spreadsheet version control, translating to an estimated 15 staff-hours saved per campaign. Given the small number of campaign cycles ($n = 4$ per condition), these results should be interpreted as preliminary effect-size estimates rather than definitive causal conclusions. The pre–post observational design without randomization cannot fully control for confounding factors such as changes in administrative staff composition or learning effects.

Regarding formal disputes, in contrast to the historical complaint rate of approximately 4.8% observed under previous processes at UEFS (14 complaints across 292 submissions over 3 years), no formal complaints were registered during the observation period (0/60 submissions). A one-tailed Fisher’s exact test comparing 0/60 vs. 14/292 yields $p = 0.069$, which does not reach conventional significance at $\alpha = 0.05$. The small sample size precludes definitive statistical conclusions. The availability of cryptographic audit logs may have contributed to discouraging disputes by providing verifiable evidence of procedural regularity [13].

As for evaluator experience, the average evaluation time per submission was $T = 8.3 \pm 2.1$ min (mean $\pm$ SD, $n = 6$ evaluators, 65 evaluations), compared to $T' = 15.2 \pm 4.7$ min under previous workflows (retrospective administrative survey, same $n = 6$ evaluators), representing a 45.4% reduction (95% CI: [26%, 59%]; Mann–Whitney $U = 2$, $p = 0.009$, two-tailed; Cohen’s $d = 1.88$, large effect). These baseline measurements derive from retrospective self-reported data and are therefore subject to recall bias; evaluators may over-

estimate prior time expenditure. Additionally, evaluator time measurements under the new system may be influenced by the Hawthorne effect, as evaluators were aware of the system’s monitoring capabilities. Technical support requests declined approximately 58% due to the improved user autonomy afforded by the system.

Finally, concerning maintenance and availability, the software is maintained as an open-source project (MIT License) on GitHub [22, 23], with continuous integration via GitHub Actions, a public issue tracker, semantic versioning, and bilingual documentation (Portuguese and English) [14]. The platform has been formally registered as a computer program with the Brazilian National Institute of Industrial Property (INPI) under Process No. 512026000776-3, securing intellectual property protection without compromising open-source availability.

6. Limitations

Study design. This is a single-institution, pre-post observational study without a randomized control group. The deployment involved a relatively small sample ($n = 60$ submissions, $n = 6$ evaluators), which limits statistical power and external generalizability. While we report Mann–Whitney U tests, confidence intervals, and effect sizes (Section 5), these should be interpreted as preliminary evidence. Multi-institutional deployments with larger samples and, ideally, randomized or quasi-experimental designs are needed to confirm the observed effects.

Baseline data quality. Comparisons with previous workflows rely on retrospective self-reported data from administrative surveys and are subject to recall bias. Evaluators may overestimate prior time expenditure, inflating the apparent efficiency gain. No objective time-tracking data exists for the pre-deployment period.

Security. As discussed in Section 3, the threat model addresses privileged insiders via RFC 3161 external timestamp anchoring. However, no formal penetration testing or external security audit has been conducted. Key management practices (JWT secrets, HMAC keys stored as environment variables, not hardcoded) follow standard recommendations but have not been independently verified. The implementation has been checked against the OWASP Top 10 [21] and `npm audit` reports zero high-severity vulnerabilities, but these do not constitute a comprehensive security assessment.

LGPD compliance. The platform provides technical controls supporting LGPD compliance (Table 4), but full compliance requires organizational policies, a formal Data Protection Impact Assessment (DPIA), and legal review

by each deploying institution. No formal DPIA has been conducted for the current deployment.

Reproducibility. Anonymized deployment metrics and analysis scripts are available at <https://doi.org/10.5281/zenodo.18868994> to support reproducibility.

7. Conclusions

avalia+Tec addresses an underserved need in institutional informatics: the transparent, procedurally accountable, and LGPD-aware governance of competitive selection processes. Preliminary deployment results from a single-institution observational study suggest that the systematic integration of well-established cryptographic techniques into a unified, open-source workflow is associated with meaningful reductions in administrative cycle time (65%, 95% CI: [55%, 73%]) and evaluator workload (45.4%, 95% CI: [26%, 59%]), while providing tamper-evident cryptographic evidence of document authenticity and procedural fairness. No formal disputes were observed during the monitoring period, though the sample size precludes definitive conclusions. The combination of hash-based integrity verification with external RFC 3161 timestamp anchoring, HMAC-authenticated scorecards, tamper-evident hash-chained audit trails, and role-based data segregation constitutes a reproducible integration engineering blueprint applicable to institutions that manage competitive selection under legal accountability requirements. The scientific contribution resides in this systematic integration rather than in the underlying cryptographic primitives. The platform is freely available under the MIT License and suitable for adoption by universities, funding agencies, and public-sector organizations. Future work will target larger-scale multi-institutional deployments with formal statistical evaluation, including randomized or quasi-experimental designs. Planned technical extensions include API integration with institutional student information systems (e.g., SIGAA, SUAP), continuous external hash anchoring through periodic blockchain checkpoint publication, and formal penetration testing with independent security audits. Identity management may be strengthened through federated authentication via SAML/OAuth 2.0 with institutional identity providers, while long-term audit storage could benefit from tiered archival strategies with cryptographic integrity seals for regulatory retention periods. Additional work will address Data Protection Impact Assessments and the development of fairness auditing modules capable of detecting systematic evaluator biases from the structured data that the platform collects. An independent external security audit is currently budgeted and planned for the

next development cycle to complement the existing automated dependency and OWASP-based assessments.

Acknowledgements

The authors acknowledge the contribution of the *avalia+* development team and the institutional support of Universidade Estadual de Feira de Santana (UEFS).

CRedit authorship contribution statement

Luiz Diego Vidal Santos: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration. **Catuxe Varjão de Santana Oliveira:** Conceptualization, Methodology, Validation, Formal analysis, Writing – review & editing, Supervision. **Dulce Paloma Vidal Santos:** Investigation, Validation. **Tânia Cristina Azevedo:** Resources, Supervision. **Renisson Neponuceno de Araújo Filho:** Writing – review & editing. **Jémison Mattos dos Santos:** Writing – review & editing. **Milton Marques Fernandes:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code and documentation are publicly available at https://github.com/ldvsantos/avalia_mais (server and web portal) and https://github.com/ldvsantos/avalia_mais_desktop (desktop application). The software is registered with INPI (Brazil) under Process No. 512026000776-3. Anonymized deployment metrics, statistical analysis scripts (Python), and the load testing framework are available in the replication package at <https://doi.org/10.5281/zenodo.18868994>.

Appendix A. Deployment and Environment Setup

This appendix provides the essential commands for deploying avalia+Tec in a production environment. The setup assumes a Linux server (Ubuntu 22.04 LTS) with PostgreSQL 12+ and Node.js 18+ installed.

```
1 #!/bin/bash
2 # Clone repository and install dependencies
3 git clone https://github.com/ldvsantos/avalia_mais.git
4 cd avalia_mais
5 npm install --production
6
7 # Configure environment variables
8 cp .env.example .env
9 # Edit .env with database credentials and JWT secret
10 # DB_HOST=localhost DB_PORT=5432
11 # DB_NAME=avalia_db DB_USER=avalia_admin
12 # JWT_SECRET=$(openssl rand -hex 32)
13
14 # Initialize PostgreSQL database with pgcrypto
15 sudo -u postgres psql -c \
16 "CREATE DATABASE avalia_db;"
17 sudo -u postgres psql -d avalia_db -c \
18 "CREATE EXTENSION IF NOT EXISTS pgcrypto;"
19 npm run db:migrate
20 npm run db:seed # optional demo data
21
22 # Start application (production mode)
23 NODE_ENV=production npm start
24 # Server listens on port 3000 by default
```

Listing 3: Server-side deployment of the avalia+Tec backend and web portal. The script clones the repository, installs dependencies, configures the database, and starts the application.

```
1 #!/bin/bash
2 # Clone desktop repository
3 git clone \
4 https://github.com/ldvsantos/avalia_mais_desktop.git
5 cd avalia_mais_desktop
6 npm install
7
8 # Configure server endpoint
9 # Edit src/main/main.js:
10 # SERVER_URL = 'https://your-server.example.com'
11
12 # Build for current platform
13 npm run build
14 # Output: dist/ directory with installer
15
16 # Cross-platform builds (requires electron-builder)
17 npx electron-builder --win --mac --linux
```

Listing 4: Building the Electron desktop application for evaluators. Produces platform-specific installers for Windows, macOS, and Linux.

References

- [1] A. Khan, R. A. Beg, M. Z. Khan, Towards automated and secure academic selection systems: A Systematic Review, *IEEE Access* 9 (2021) 89234–89256. doi:10.1109/ACCESS.2021.3089076.
- [2] A. Esposito, A. Sangrà, M. F. Maina, Towards a characterization of the role of Web-Based Technologies in Higher Education, *Educational Technology & Society* 19 (4) (2016) 44–57.
- [3] R. F. do Brasil, Lei geral de proteção de dados pessoais (LGPD), *diário Oficial da União*, Brasília (2018).
- [4] A. Cavoukian, Privacy by design: The 7 Fundamental principles, Information and Privacy Commissioner of Ontario Available at: https://www.ipc.on.ca/wp-content/uploads/2016/08/Foundational_Principles.pdf (2011).
- [5] M. C. K. Machado, C. Machado, D. T. d. Souza, O uso de tecnologias digitais na gestão acadêmica de instituições de ensino superior, *Revista Brasileira de Educação* 23 (2018) 1–20. doi:10.1590/S1413-24782018230038.
- [6] Universidade Federal do Rio Grande do Norte, SIGAA – Sistema Integrado de Gestão de Atividades Acadêmicas, accessed: 2024-12-01 (2023).
URL https://www.info.ufrn.br/wikisistemas/doku.php?id=suporte:sigaa:visao_geral
- [7] A. Soergel, A. Saunders, A. McCallum, Open scholarship and peer review: A time for experimentation, in: *ICML 2013 Workshop on Peer Reviewing and Publishing Models*, 2013.
- [8] EasyChair, EasyChair conference management system, accessed: 2024-12-01 (2024).
URL <https://easychair.org/>
- [9] E. Kohler, HotCRP: A conference review tool, accessed: 2024-12-01 (2004).
URL <https://hotcrp.com/>
- [10] CAPES/MEC, Plataforma Sucupira – Coleta de dados, Avaliação e Gestão de Programas de Pós-Graduação, accessed: 2024-12-01 (2024).
URL <https://sucupira.capes.gov.br/>

- [11] S. J. Piotrowski, G. G. Van Ryzin, Citizen attitudes toward transparency in government, *The American Review of Public Administration* 37 (3) (2007) 306–323. doi:10.1177/0275074006296777.
- [12] L. Marques, G. da Silva, M. Oliveira, Digital governance and transparency in Public Universities: A Brazilian Case Study, *Government Information Quarterly* 39 (1) (2022) 101635. doi:10.1016/j.giq.2021.101635.
- [13] C. Hood, Public management: The third electoral wave?, *Public Administration* 69 (4) (2007) 405–427.
- [14] I. Sommerville, *Software Engineering*, 10th Edition, Pearson, 2015.
- [15] I. O. for Standardization (ISO), *Information security management systems — Requirements* (2022).
- [16] N. I. of Standards, T. (NIST), *Descriptions and rules for use of the SHA-2 family of Secure Hash Algorithms* (2012).
- [17] M. Bellare, H. Krawczyk, Keying hash functions for message authentication, in: *Advances in Cryptology — CRYPTO '96*, Springer, Berlin, Germany, 1997, pp. 1–15. doi:10.1007/3-540-68339-9_1.
- [18] C. Adams, P. Cain, D. Pinkas, R. Zuccherato, Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP), RFC 3161 (Aug. 2001). doi:10.17487/RFC3161.
URL <https://www.rfc-editor.org/rfc/rfc3161>
- [19] G. McGraw, *Software Security: Building Security In*, Addison-Wesley Professional, 2006.
- [20] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation), *Official Journal of the European Union* L 119, pp. 1–88 (2016).
- [21] O. Foundation, OWASP Top 10 web application security risks Available at: <https://owasp.org/www-project-top-ten/> (2023).
- [22] L. D. Vidal, E. avalia+, avalia+tec: Govern Selection Processes With Cryptographic Integrity, gitHub Repository (2026).
URL https://github.com/ldvsantos/avalia_mais

- [23] L. D. Vidal, E. avalia+, avalia+tec Desktop: Distributed Application Electron, gitHub Repository (2026).
URL https://github.com/ldvsantos/avalia_mais_desktop